Question 3: Set up monitoring using Prometheus and visualize metrics using Grafana dashboards for a target application or system.

The Flask app exposes Prometheus counters on /metrics. A three-container monitoring stack (app + prometheus + grafana) was brought up with Docker Compose, Grafana was provisioned from files with a dashboard, and live traffic was visualized.

Prometheus configuration (monitoring/prometheus.yml)

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: "lab3-app"
    metrics_path: /metrics
    static_configs:
      - targets: ["app:5000"]

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

App instrumentation (excerpt)

```
from prometheus_client import Counter, generate_latest

REQUEST_COUNT = Counter("lab3_app_requests_total",
                        "Total HTTP requests", ["endpoint", "status"])

@app.after_request
def count_request(response):
    REQUEST_COUNT.labels(request.path, str(response.status_code)).inc()
    return response

@app.route("/metrics")
def metrics():
    return generate_latest(), 200, {"Content-Type": CONTENT_TYPE_LATEST}
```

Verification

```
$ curl -s http://localhost:5003/metrics | grep lab3_app
lab3_app_requests_total{endpoint="/",status="200"} 60.0
lab3_app_requests_total{endpoint="/health",status="200"} 60.0
lab3_app_requests_total{endpoint="/beta",status="404"} 60.0

$ curl -s http://localhost:9090/api/v1/targets | jq targets
{"job":"lab3-app","health":"up"}
{"job":"prometheus","health":"up"}
```

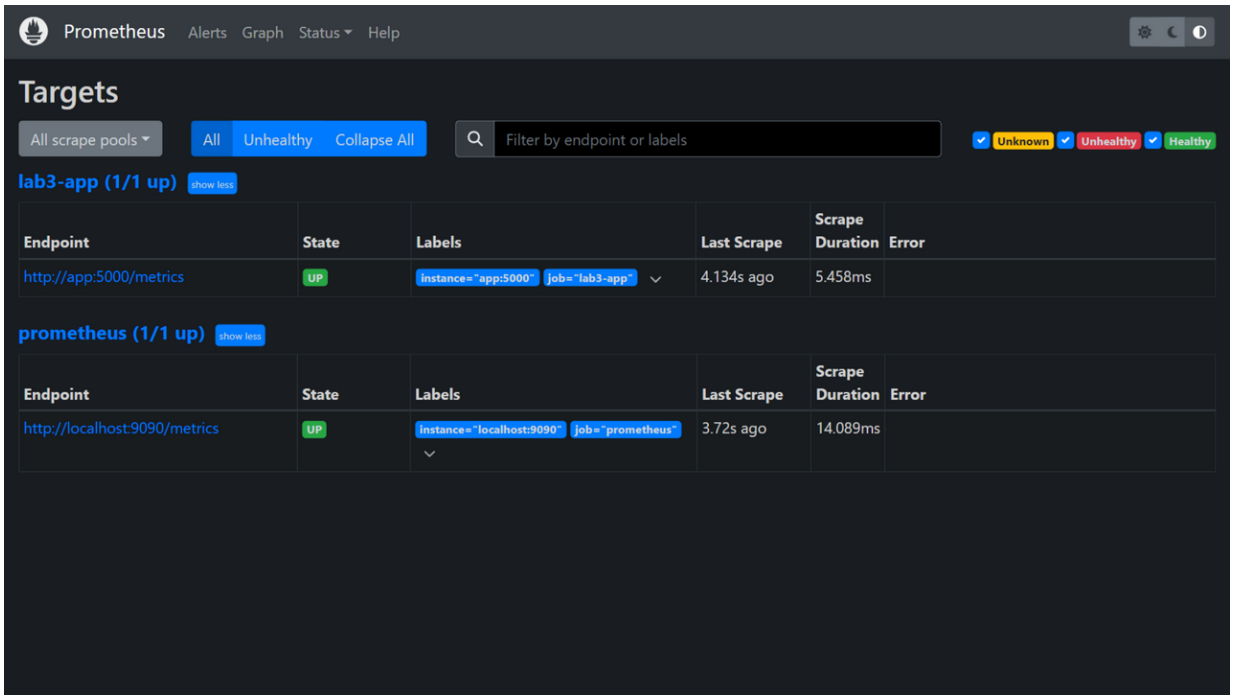


Fig: Prometheus targets page - both targets UP



Fig: Grafana dashboard - live request rate, totals and uptime

Observations

- The Grafana datasource and dashboard are provisioned from files on startup - zero manual setup.
- The after_request hook counts every response with endpoint + status labels, so errors are visible per endpoint (the /beta 404s during the flag demo show up as their own series).
- Prometheus scrapes every 5 seconds; rate() over 1m windows smooths the counters into req/s.
- The monitoring stack is a single docker compose file (app, prometheus, grafana on one network).

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Ansible, Terraform, Prometheus, Grafana, the ELK stack and Selenium. The evidence (command logs, configuration files, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-3 under the docs/ directory.